

Loan module: presentation, workflow, and UX recommendations

This document compares the **legacy FundFlow** loan implementation with **FundFlow SaaS**, and recommends how to present loans in terms of functionality, workflow, and UI/UX—without copying unnecessary complexity from the old project.

Audience: product, operations, and developers planning loan module work.

Related code (SaaS): App\Services\LoanService, App\Support\LoanSettings, App\Filament\Tenant\Pages\LoanQueue, App\Filament\Resources\LoanResource, App\Filament\Resources\LoanResource\Pages, App\Filament\Resources\LoanResource\Widgets, App\Filament\Tenants\LoanFilamentTenancy.
loan Filament resources under app/Filament/Tenant/Resources/Loans/.

Related code (legacy): docs/loan-lifecycle-human-readable-guide.md, docs/loan-lifecycle-workflows-and-accounting in the old repository.

1. Where each project sits today

Dimension	Legacy FundFlow	FundFlow SaaS (current)
Mental model	Installment loan with tiers, queue, guarantor, partial disburse	Application → approve → ledger steps → lump repayments
Admin focus	Loan queue + tier buckets + installment grid	Loans table + simple queue + row actions
Member focus	Apply, calculator, installments, guaranteed loans	Loan history + apply modal
Settings	Many keys (tiers, late fees, grace, auto-allocate)	Settings → Loans tab (eligibility + defaults)
Repayment	Schedule rows, cron, mark paid, early settle	Manual recordRepayment ; no schedule UI
Strength	Operations-ready for a real mutual fund	Ledger-correct, testable, multi-tenant
Weakness	Heavy; easy to lose the thread	Thin UX; disburse vs payout is correct in books but opaque in UI

The legacy app optimized for **fund operators who work in loans daily**. SaaS optimized for **correct double-entry**. The recommended path: **keep the SaaS ledger model; improve the story told in the UI**.

2. Functionality: keep, add, or defer

2.1 Keep from SaaS (do not regress)

- **Explicit ledger phases** — fund allocation → credit member cash → bank payout (aligns with bank import and audit).
- **Settings-driven eligibility** — membership months, minimum fund balance, borrow multiplier (and optional absolute cap).
- **Single active loan rule** — appropriate for most mutual funds.
- **Reject / cancel with recorded reasons** — required for member trust and operations audit.

2.2 Bring back selectively from legacy (high value, bounded scope)

Feature	Why	SaaS mapping
Repayment schedule (display)	Answers “what is due this month?”	Generate rows from flat interest + term; show paid/pending without full installment engine on day one
Early payoff	Common operational request	Single action: <code>recordRepayment(outstanding)</code> with confirmation
State-change notifications	Reduces support load	~6–7 templates: submitted, approved, rejected, funded, payout complete, repayment, completed
Member loan calculator (read-only)	Fewer invalid applications	Reuse <code>LoanService::computeMonthlyRepayment()</code> + <code>LoanSettings::maxLoanAmountForMember()</code> before apply

2.3 Defer or redesign from legacy (high cost)

Legacy feature	Recommendation
Loan tiers + fund tiers + queue resequencing	Use FIFO queue + optional emergency pin unless the fund truly caps lending by pool band
Partial disbursements (<code>LoanDisbursement</code> tranches)	Only if the fund routinely releases loans in chunks
Guarantors + witnesses	Add only when legal/process requires
Late fees, default, guarantor debit	Phase 2: needs delinquency jobs + written policy
CSV loan import	Low priority vs member apply + queue

2.4 Additions neither project does well today

- **Loan health on member profile** — one card: status, outstanding, next due, % repaid (mirror account insights widgets).
- **Contribution → loan allocation** — optional setting to apply posted contributions toward loan repayment (legacy had auto-allocate flags).

3. Workflow: clearer pipeline

3.1 Problem in current SaaS UI

Admins see: `pending` → `approved` → `disbursed` → `repaying` → `completed`, plus separate actions **Disburse** and **Payout**. That is **five statuses and two verbs** for “money reached the member,” which confuses non-accountants.

3.2 Recommended user-facing stages

Present **four member-visible stages**; map ledger statuses internally:

User-facing	Internal status / actions
Applied	<code>pending</code>
Approved	<code>approved</code> (terms locked)
Funded	<code>disbursed</code> ; sub-step Sent to bank when payout recorded
Repaying	<code>repaying</code>
Closed	<code>completed</code> , <code>rejected</code> , or <code>cancelled</code>

3.3 Admin stepper on loan view

Use a horizontal stepper on the loan **view** page (similar to membership application insights):

[Applied] → [Approved] → [Allocate to ledger] → [Bank payout] → [Repaying] → [Closed]

Step label (UI)	Current action	Accounting meaning
Allocate to ledger	Disburse	Debit master fund + member fund; credit member cash
Bank payout	Payout	Debit member cash + master cash (match bank statement later)

Members should see simplified copy, e.g. **Approved** — awaiting transfer then **Active** — repay monthly, not raw **disbursed** / **repaying**.

3.4 Queue vs full list

Loan queue (default under Loans navigation) with tabs:

1. **Needs decision** — pending
2. **Ready to fund** — approved
3. **Awaiting bank payout** — disbursed (payout not yet recorded)
4. Badge counts on nav (already started on loans list header)

All loans — searchable archive with filters.

Sort: `applied_at` FIFO; optional **emergency** boolean to pin to top (avoid full tier system unless required).

3.5 Member workflow

Check eligibility (dashboard)

- Calculator (optional)
- Apply (purpose + amount; terms from settings)
- Track status (timeline + rejection reason)
- When active: next payment + outstanding (schedule when built)
- **Cancel** only while pending.
- **Rejection reason** visible on list and detail, not only on view.

4. UI/UX patterns (aligned with SaaS conventions)

SaaS already uses **Applications** and **account insights** (`ff-app-insights`, hero, KPI strip). Loans should follow the same language.

4.1 Tenant admin

Pattern	Description
Loans index insights widget	KPIs: pending count, approved awaiting fund, portfolio outstanding, repayments this month; hero CTA → queue
Loan view as command center	Stepper + one primary next action ; terms card; repayments table; link to ledger transactions
View-first	Row actions secondary; primary flow from view page
Settings → Loans	Sections: Eligibility, Product defaults, Repayment (later); live preview of max amount and monthly installment

Pattern	Description
Member context	Loans on member profile show outstanding + status; link to loan view

4.2 Member portal

Pattern	Description
Dashboard card	Extend MyFundOverview : eligible → apply CTA; active loan → % repaid, next due, outstanding
My Loans list	Human status labels; amount + progress bar per row
Loan detail	Timeline; schedule when available; rejection callout at top
Apply flow	Prefer 3-step wizard (Amount → Purpose → Review) over one large modal; show schedule preview on review

4.3 Terminology (reduce cognitive load)

Avoid in UI	Prefer
Disburse / Payout	Allocate to member account / Send to bank
Repaying	Active loan
Approved (to members)	Approved — not yet funded
Record repayment	Post payment

Use the same strings in English and `lang/ar.json` (bilingual UI rule).

5. Phased implementation roadmap

Phase A — UX clarity (no new database tables)

- Loan view stepper + single primary action button
- Queue tabs (needs decision / ready to fund / awaiting payout)
- Member timeline + dashboard loan card
- **LoansInsightsWidget** on list (mirror applications / accounts)
- Rename actions and status labels per table above

Phase B — Operational completeness

- Amortization schedule (generated, read-only)
- Record repayment from loan view; default amount = monthly installment
- Early payoff action
- Email / in-app notifications on state changes

Phase C — Only if fund rules require

- Emergency priority in queue
- Auto-allocate contributions to loan repayment
- Delinquency, late fees, guarantor flows (policy + jobs)

6. Summary

	Legacy	SaaS today	Target
Accounting	Complex installments + tiers	Clear multi-leg disburse/payout/repay	Keep SaaS ledger
Operations	Queue, schedule, calculator	Table + basic queue	Queue tabs + schedule display
Members	Rich self-service	Apply + view	Wizard + timeline + dashboard card
UI	Many bespoke widgets	CRUD + actions in dropdown	Insights + view-first command center

Bottom line: Keep SaaS accounting and settings; add legacy **visibility** (schedule, queue structure, calculator, notifications); avoid legacy **tier/partial-disburse** complexity unless fund bylaws require it; present loans with the same polish as Applications and account insights.

7. Current SaaS implementation reference (baseline)

As of the loan workflow implementation in this repository:

- **Settings:** Settings page → **Loans** tab → LoanSettings
- **Queue:** LoanQueue Filament page
- **Actions:** LoanTableActions (approve, reject, disburse, payout, record repayment, cancel)
- **Service:** LoanService — eligibility, apply, approve, reject, cancel, disburse, payout, recordRepayment
- **Statuses:** pending, approved, rejected, cancelled, disbursed, repaying, completed, defaulted (reserved)

This document describes **recommended next steps**; items in Phases A–C are not all implemented yet.